ABSTRACCIÓN Y USO DE DATOS

3CM30

Reporte de prácticas

Murata Abreu Rodrigo Yael

Van Scoit Urtiz Ian Aaron

1. HOLA MUNDO

EL siguiente código se hizo con la estructura de archivos solicitada por el maestro.
-Programa Principal.
-Programa de Funciones.
-Librería.

-Se uso una clase “Saludo”
-Se uso una función “Hola”
Se Utilizo una variable de tipo string para poder imprimir el nombre del usuario, para después mostrarlo con un cordial saludo.
Solo se presentó un error y fue crear archivos independientes en lugar de un proyecto y en el proyecto agregar los archivos. Por consecuente se corrigió este error y se realizó con éxito la compilación y corrimiento del programa.

Project Classes Debug

Project_Holamundo
- F_p1.cpp
- Lib_p1.h
- main_p1.cpp

F_p1.cpp | Lib_p1.h | main_p1.cpp

```cpp
#include<iostream>
#include "Lib_p1.h"

using namespace std;

int main(int argc, char** argv) {
    Saludo a= Saludo();
    a.Hola();

    cout << "TAMAÑO DE UN char: " << sizeof(char) << " bytes" << endl << endl;
    cout << "TAMAÑO DE UN float: " << sizeof(float) << " bytes" << endl << endl;
    cout << "TAMAÑO DE UN boolean: " << sizeof(bool) << " bytes" << endl << endl;
    cout << "TAMAÑO DE UN double: " << sizeof(double) << " bytes" << endl << endl;
    cout << "TAMAÑO DE UN long: " << sizeof(long) << " bytes" << endl << endl;
    cout << "TAMAÑO DE UN short: " << sizeof(short) << " bytes" << endl << endl;


    return 0;


}
```

Compiler (2) | Resources | Compile Log | Debug | Find Results | Console | Close

Abort Compilation

Shorten compiler patl

```
Compilation results...
--------
- Errors: 0
- Warnings: 0
- Output Filename: D:\IPN 2025\TERCER SEMESTRE\ABSTRACCION Y USO DE DATOS\EMBARCADERO_DEV\Project_Holamundo.e
- Output Size: 2.99638748168945 MiB
- Compilation Time: 3.42s
```

D:\IPN 2025\TERCER SEMESTI

```
Ingresa tu nombre por favor UPIICSIANO:
Rodrigo
HOLA Rodrigo UPIICSA TE DA LA BIENVENIDA....
```

2. SUMA DE DOS VALORES COMO PARÁMETROS

EL siguiente código se hizo con la estructura de archivos solicitada por el maestro.
-Programa Principal.
-Programa de Funciones.
-Librería.

-Se uso una clase “Operaciones”
-Se declaró suma con dos parámetros.
-Se utilizaron variables de tipo int para poder realizar la suma, y después mostrar el resultado.
No se presentó ningún error.

ug main_suma.cpp F_suma.cpp Lib_suma.h

```cpp
#include <iostream>
#include "Lib_suma.h"

using namespace std;

int main(){

    Operaciones op;
    int a, b, resultado;

    cout << "Ingresa el primer numero: ";
    cin >> a;

    cout << "Ingresa el segundo numero: ";
    cin >> b;

    resultado = op.suma(a,b);

    cout << "La suma es: " << resultado << endl;

    return 0;
}
```

sources Compile Log Debug Find Results Console Close

```
Compilation results...
--------
- Errors: 0
- Warnings: 0
```

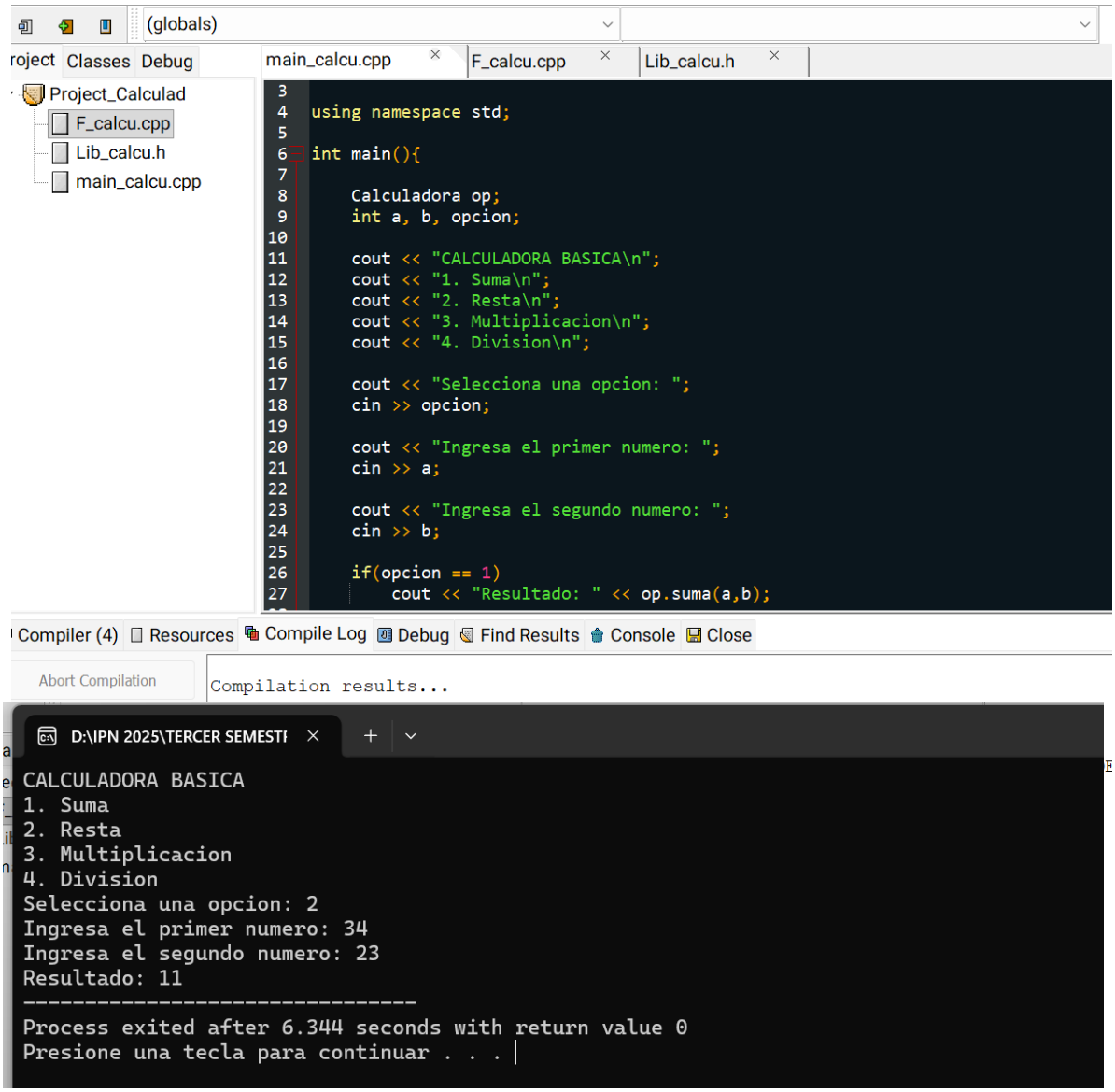
D:\IPN 2025\TERCER SEMESTI

```
Ingresa el primer numero: 10
Ingresa el segundo numero: 9
La suma es: 19

--------------------------------
Process exited after 4.684 seconds with return value 0
Presione una tecla para continuar
```

3. CALCULADORA BÁSICA(SUMA, RESTA, DIV, MULT)

EL siguiente código se hizo con la estructura de archivos solicitada por el maestro.
-Programa Principal.
-Programa de Funciones.
-Librería.

-Se uso una clase “Calculadora”
-Se declararon suma, resta, mult y div como parámetros, donde div fue el único float
En el main principal se realizó un menú donde puedes seleccionar la operación que desees, y después mostrar el resultado.
Se presentó el error de declarar la división de int y que después de hacer la operación nos retornará un entero, esto no es posible por lo que se corrigió usando un return static_cast<float>(a) / b; para tener nuestra divison con decimales



4. CALCULADORA CON 2,0,3 PARAMETROS(SUMA, RESTA, DIV, MULT)

EL siguiente código se hizo con la estructura de archivos solicitada por el maestro.
-Programa Principal.
-Programa de Funciones.
-Librería.

-Se uso una clase “Calculadora” con las mismas funciones que la anterior.
-Se declararon suma, resta, mult y div como parámetros 0, 2 y 3, donde div fue el único float
En el main principal se realizó un menú donde puedes seleccionar la operación que desees, y después mostrar el resultado.
Se presentó el error en los parámetros pues al principio no usamos lo que se usa en C++, la sobrecarga de funciones (function overloading).
Es decir, varias funciones con el mismo nombre pero diferente número de parámetros.

ect Classes Debug

Project_calcu-param
- F_calcup.cpp
- Lib_calcup.h
- main_calcup.cpp

main_calcup.cpp | F_calcup.cpp | Lib_calcup.h

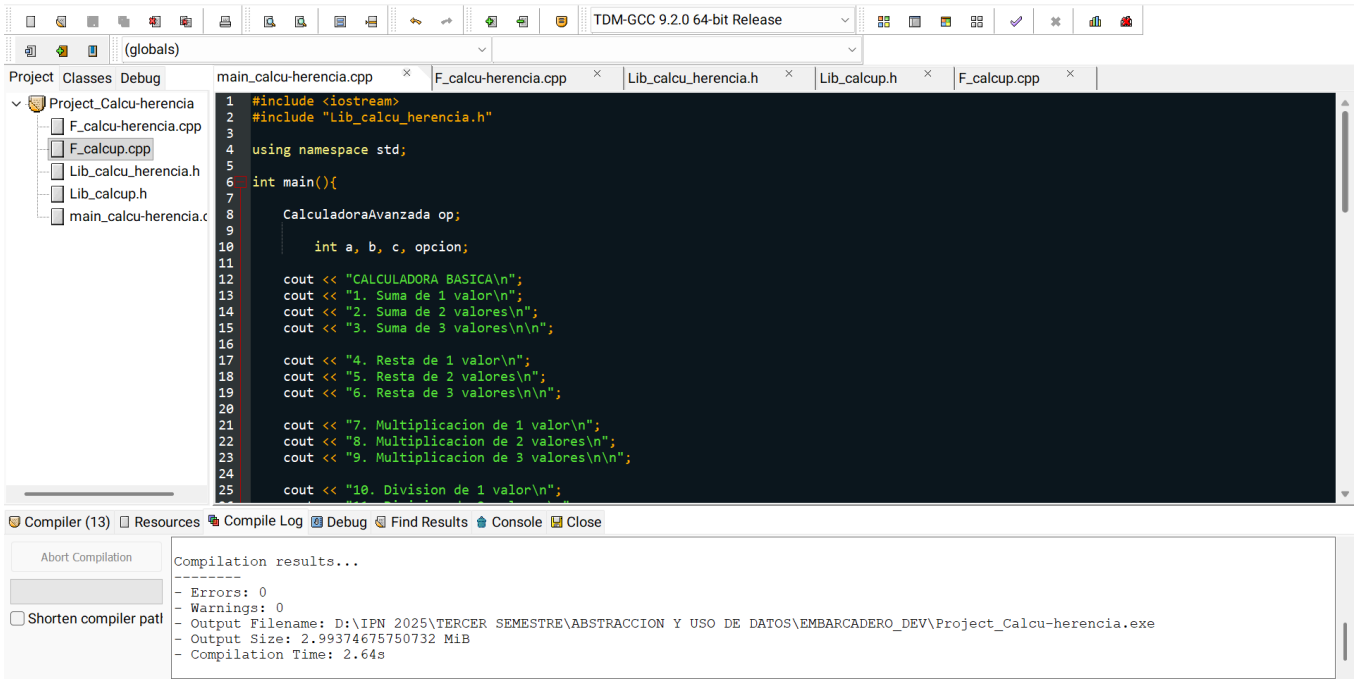
```cpp
#include <iostream>
#include"Lib_calcup.h"

using namespace std;

int main(){

    Calculadora op;
    int a, b, c, opcion;

    cout << "CALCULADORA BASICA\n";
    cout << "1. Suma de 1 valor\n";
    cout << "2. Suma de 2 valores\n";
    cout << "3. Suma de 3 valores\n\n";

    cout << "4. Resta de 1 valor\n";
    cout << "5. Resta de 2 valores\n";
    cout << "6. Resta de 3 valores\n\n";

    cout << "7. Multiplicacion de 1 valor\n";
    cout << "8. Multiplicacion de 2 valores\n";
    cout << "9. Multiplicacion de 3 valores\n\n";

    cout << "10. Division de 1 valor\n";
    cout << "11. Division de 2 valores\n";
```

```
7. Multiplicacion de 1 valor
8. Multiplicacion de 2 valores
9. Multiplicacion de 3 valores

10. Division de 1 valor
11. Division de 2 valores
12. Division de 3 valores

Selecciona una opcion: 2
Ingresa el primer numero: 3
Ingresa el segundo numero: 4
Ingresa el Tercer numero: 5
Resultado: 7
```

5. CALCULADORA NUEVA CON HERENCIA (SUMA, RESTA, DIV, MULT, POT,MOD)

EL siguiente código se hizo con la estructura de archivos solicitada por el maestro.
-Programa Principal.
-Programa de Funciones.
-Librería.

-Se uso una clase BASE “Calculadora” y una clase Heredada “CalculadoraAvanzada” con las mismas funciones que la anterior, sin embargo a esta nueva clase le agregamos potencia y modular.
En el main principal se realizó un menú donde puedes seleccionar la operación que desees, y después mostrar el resultado.
Se presentó el error en los archivos pues al principio no colocamos los archivos de la clase base y como consecuencia pues no leía nuestras funciones el compilador, La solución fue muy sencilla, le añadimos al proyecto los archivos necesarios y listo.



6. CALCULADORA NUEVA CON HERENCIA Y SOBREESCRITURA DE FUNCIONES (SUMA, RESTA, DIV, MULT)

EL siguiente código se hizo con la estructura de archivos solicitada por el maestro.
-Programa Principal.
-Programa de Funciones.
-Librería.

-Se uso una clase BASE “Calculadora” y una clase Heredada “CalculadoraNueva” con las mismas funciones que la anterior, sin embargo a esta nueva clase le haremos una sobreescritura a las funciones de multiplicar y dividir.
En el main principal se realizó un menú donde puedes seleccionar la operación que desees, y después mostrar el resultado.
Se presentó el error en los archivos pues al principio no colocamos los archivos de la clase base y como consecuencia pues no leía nuestras funciones el compilador, La solución fue muy sencilla, le añadimos al proyecto los archivos necesarios y listo.

```cpp
#include <iostream>
#include"Lib_calcu-ow.h"

using namespace std;

int main(){

    CalculadoraNueva op;
    int a, b, opcion;

    cout << "CALCULADORA BASICA\n";
    cout << "1. Suma\n";
    cout << "2. Resta\n";
    cout << "3. Multiplicacion\n";
    cout << "4. Division\n";

    cout << "Selecciona una opcion: ";
    cin >> opcion;

    cout << "Ingresa el primer numero: ";
    cin >> a;

    cout << "Ingresa el segundo numero: ";
    cin >> b;
```

```
D:\IPN 2025\TERCER SEMESTI
CALCULADORA BASICA
1. Suma
2. Resta
3. Multiplicacion
4. Division
Selecciona una opcion: 3
Ingresa el primer numero: 3
Ingresa el segundo numero: 3
Resultado: 9
--------------------------------
Process exited after 6.517 seconds with
```

```cpp
#include "Lib_calcu-ow.h"

int CalculadoraNueva::multiplicacion(int a, int b){

    int resultado = 0;

    for(int i = 0; i < b; i++){
        resultado = resultado + a;
    }

    return resultado;
}

int CalculadoraNueva::division(int a, int b){

    int contador = 0;

    while(a >= b){
        a = a - b;
        contador++;
    }

    return contador;
```

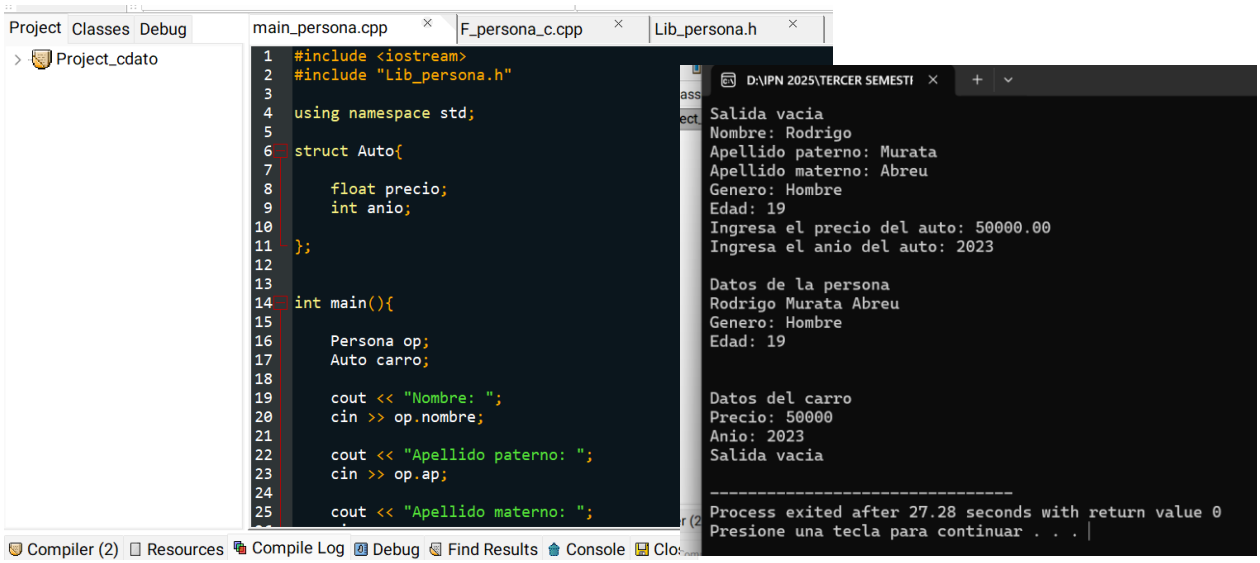
7. CREACION DE TIPO DE DATO EN PE Y PO

EL siguiente código se hizo con la estructura de archivos solicitada por el maestro.
-Programa Principal.
-Programa de Funciones.
-Librería.

-Se uso una clase “Persona”
-Se uso una struct “Auto”

-Se uso un Constructor y un destructor
-Se Utilizo una variable de tipo string para poder imprimir el nombre, ap, am y genero del usuario, para después mostrarlo.
-Se utilizo int para edad y año.
-Se utilizo un float para precio.

Solo se presentaron algunos errores y fueron: ponerle a nuestros constructores y destructores un void; además, de no llamar a nuestra clase persona con la variable (op.nombre), puesto que nuestras variables están declaradas en esa clase. Por consecuente se corrigieron estos errores y se realizó con éxito la compilación y corrimiento del programa.



8. IDENTIFICAR EL TAMAAÑO DE PE Y POO

EL siguiente código se hizo con la estructura de archivos solicitada por el maestro.
-Programa Principal.
-Programa de Funciones.
-Librería.

-Se uso el programa anterior
-SE CONTARON LOS BYTES CON SIZEOF
NO SE PRESENTÓ NINGUN ERROR DE COMPILACIÓN.

```cpp
#include <iostream>
#include "Lib_persona_bytes.h"

using namespace std;

struct Auto{

    float precio;
    int anio;

};


int main(){

    Persona op;
    Auto carro;

    cout << "Nombre: ";
    cin >> op.nombre;

    cout << "Apellido paterno: ";
    cin >> op.ap;

    cout << "Apellido materno: ";
```

```
Salida vacia
Nombre: Rodrigo
Apellido paterno: Murata
Apellido materno: Abreu
Genero: Hombre
Edad: 19
Ingresa el precio del auto: 400000
Ingresa el anio del auto: 2025

Datos de la persona
Rodrigo Murata Abreu
Genero: Hombre
Edad: 19


Datos del carro
Precio: 400000
Anio: 2025
Struct auto: 8 bytes
Class Persona: 136 bytes
Salida vacia

--------------------------------
Process exited after 17.48 seconds with return value 0
Presione una tecla para continuar . . .
```

9. PROMEDIO

EL siguiente código se hizo con la estructura de archivos solicitada por el maestro.
-Programa Principal.
-Programa de Funciones.
-Librería.

-Se uso una clase “Promedio”
-Se declararon las variables y funciones correspondientes, así como la asignación de sus parámetros.
Se Utilizo una variable de tipo int y float , para después mostrar las mismas.
NO SE PRESENTÓ NINGUN ERROR

(globals)

Project Classes Debug

Project_promedio

main_promedio.cpp | F_promedio.cpp | Lib_promedio.h | F_calcup.cpp

```cpp
#include <iostream>
#include "Lib_promedio.h"

using namespace std;

int main(){

    Promedio op;

    int a,b,c,d,e;

    cout << "Ingresa tus 5 calificaiones redondedadas (ej. 10,9,8)\n";

    cout << "1er Calif:: ";
    cin >> a;

    cout << "2nd Calif:: ";
    cin >> b;

    cout << "3th Calif:: ";
    cin >> c;

    cout << "4th Calif:: ";
    cin >> d;

```

D:\IPN 2025\TERCER SEMESTI

```
Ingresa tus 5 calificaiones redondedadas (ej. 10,9,8)
1er Calif:: 10
2nd Calif:: 8
3th Calif:: 9
4th Calif:: 8
5th Calif:: 7

Resultados
Suma: 42
Promedio: 8.4
Media: 8.4
Maxima Calif: 10
Minima Calif: 7

--------------------------------
Process exited after 10.42 seconds with return value 0
Presione una tecla para continuar . . .
```

10. PROMEDIO CON ARRAY

EL siguiente código se hizo con la estructura de archivos solicitada por el maestro.
-Programa Principal.
-Programa de Funciones.
-Librería.

-Se uso una clase “Promedio”
-Se declararon las variables y funciones correspondientes, así como la asignación de sus parámetros (en los parametros se utilizó int v[] para cada elemento del array e int n, que señala el numero total de elementos del array).
-Se declaró un arreglo números que contendrá las 5 calif.
Se Utilizo una variable de tipo int y float , para después mostrar las mismas.
Se presentaron varios errores, entre estos, confundir los parametros v[], y n, pensando que v[] era el total de elementos y n cada elemento, además de que en el main principal utilizamos números[i], en lugar de v[], y en los cout poner al revess (números,5)

Project Classes Debug

Project_Promedio_array

main_promedio_array.cpp | F_promedio_array.cpp | Lib_promedio_array.h

```cpp
    Promedio op;
    int numeros[5];

    cout << "Ingresa 5 Calif:\n";

    for(int i = 0; i < 5; i++){
        cout << "Calif: " << i+1 << ": ";
        cin >> numeros[i];
    }

    cout << "\nResultados\n";

    cout << "Suma: " << op.suma(numeros,5) << endl;

    cout << "Promedio: " << op.promedio(numeros,5) << endl;

    cout << "Media: "<< op.promedio(numeros,5) << endl;

    cout << "Maximo: "<< op.maximo(numeros,5) << endl;

    cout << "Minimo: "<< op.minimo(numeros,5) << endl;

    return 0;
}
```

Compiler (4) Resources Compile Log Debug Find Results Console Close

D:\IPN 2025\TERCER SEMESTI

```
Ingresa 5 Calif:
Calif: 1: 10
Calif: 2: 9
Calif: 3: 8
Calif: 4: 7
Calif: 5: 6

Resultados
Suma: 40
Promedio: 8
Media: 8
Maximo: 10
Minimo: 6

--------------------------------
Process exited after 5.397 seconds with return value 0
Presione una tecla para continuar . . .
```

11. PROMEDIO CON ARRAY CON PUNTERO

EL siguiente código se hizo con la estructura de archivos solicitada por el maestro.
-Programa Principal.
-Programa de Funciones.
-Librería.

-Se uso una clase “Promedio”
-Se declararon las variables y funciones correspondientes, así como la asignación de sus parámetros (en los parametros se utilizó int v[] para cada elemento del array e int n, que señala el numero total de elementos del array).
-Se declaró un arreglo números que contendrá las 5 calif.
Se Utilizo una variable de tipo int y float , para después mostrar las mismas.
Se presentaron varios errores, entre estos, confundir los parametros v[], y n, El puntero declarado como p en el cout poníamos p[i], cuando en realidad era p+i.

```
using namespace std;

int main(){

    Promedio op;

    int numeros[5];
    int *p;

    p = numeros;

    cout << "Ingresa 5 Calif: \n";

    for(int i = 0; i < 5; i++){
        cout << "Calif: " << i+1 << ": ";
        cin >> *(p+i);
    }

    cout << "\nResultados\n";

    cout << "Suma: " << op.suma(p,5) << endl;

    cout << "Promedio: " << op.promedio(p,5) << endl;

    cout << "Media: " << op.promedio(p,5) << endl;
```

```
Ingresa 5 Calif:
Calif:  1: 10
Calif:  2: 9
Calif:  3: 8
Calif:  4: 7
Calif:  5: 6

Resultados
Suma: 40
Promedio: 8
Media: 8
Maximo: 10
Minimo: 6

--------------------------------
Process exited after 5.282 seconds with return value 0
Presione una tecla para continuar . . .
```

12. MULTIPLICACION DE MATRICES

El siguiente código se realizó utilizando la estructura de archivos solicitada por el maestro:

P-rograma Principal.

-Programa de Funciones.

-Librería.

$$R[i][j] = \sum_{k=0}^{n-1} A[i][k] \times B[k][j]$$

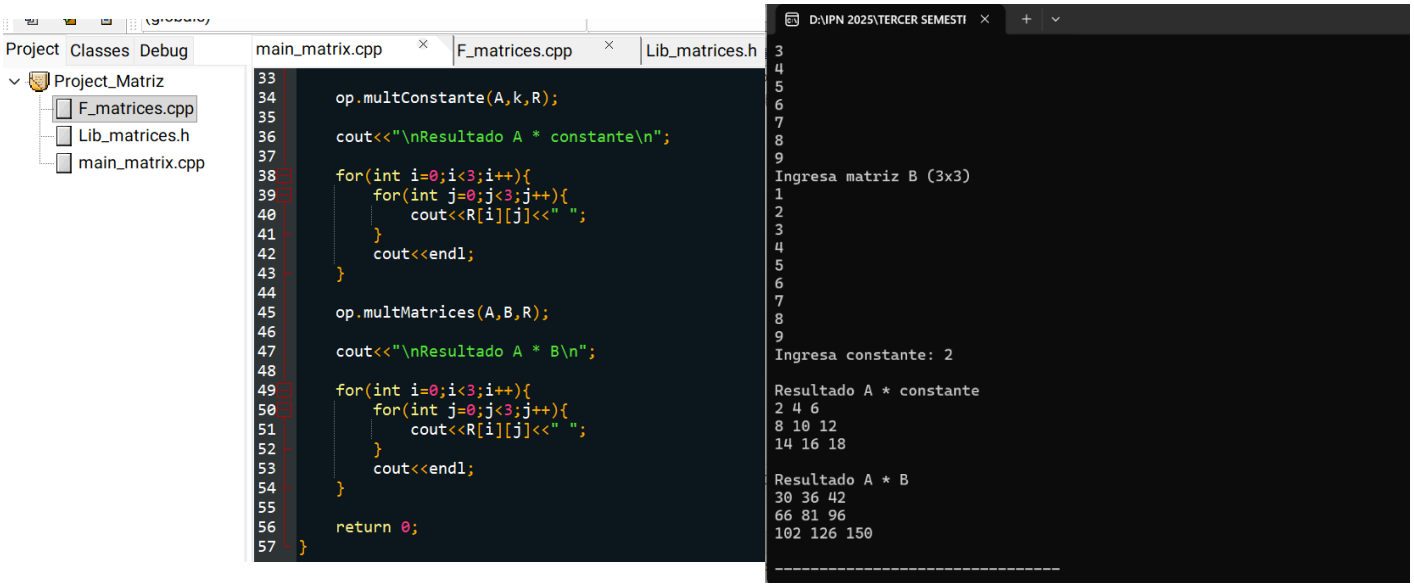
Se utilizó una clase llamada “Matrices”, en la cual se declararon las funciones necesarias para realizar las operaciones solicitadas. Se declararon las funciones: multConstante(int A[3][3], int k, int R[3][3]) || multMatrices(int A[3][3], int B[3][3], int R[3][3])

En la operación A * constante, cada elemento de la matriz A se multiplica por una constante k, almacenando el resultado en la matriz R.

En la operación A * B, se realiza la multiplicación de matrices, donde cada elemento de la matriz resultado se obtiene multiplicando la fila correspondiente de la matriz A por la columna correspondiente de la matriz B, sumando los productos obtenidos.

Durante la elaboración del programa se presentaron algunos errores, entre ellos:

Confundir los índices de las matrices en los ciclos for. No inicializar en 0 los valores de la matriz resultado antes de realizar la multiplicación de matrices. Confundir el uso de la variable k, ya que se utiliza tanto como constante como índice dentro del ciclo interno de la multiplicación de matrices.



13. LISTA DE PERSONAS Y COCHES

El siguiente código se realizó utilizando la estructura de archivos solicitada por el maestro:

- Programa Principal.
- Programa de Funciones.
- Librería.

Se utilizó una clase llamada “Registro”, en la cual se declararon las funciones necesarias para capturar y mostrar la información de personas y autos.

Se crearon dos clases principales:

- Persona, que contiene los datos: nombre, apellido paterno, apellido materno, género y edad.
- Auto, que contiene los datos: precio y año.

Para almacenar la información se utilizaron arreglos de objetos, donde se genera una lista de personas y una lista de coches.

En el programa principal se declararon:

- Un arreglo de tipo Persona para guardar la lista de personas.
- Un arreglo de tipo Auto para guardar la lista de autos.

Se utilizaron ciclos for para capturar la información de cada elemento de las listas, solicitando los datos al usuario mediante cin. Posteriormente se utilizaron otros ciclos para mostrar la información almacenada mediante cout.

Durante el desarrollo del programa se presentaron algunos errores comunes, entre ellos:

- Confundir la forma de acceder a los atributos de las clases al utilizar arreglos de objetos.
- Olvidar indicar correctamente el número de elementos que se capturan en las funciones.
- Confundir la sintaxis al mostrar los datos almacenados dentro de los arreglos.

Finalmente, el prog rama permite registrar y mostrar una lista de personas y una lista de coches, organizando la información mediante clases y arreglos, respetando la estructura de archivos solicitada.

Project Classes Debug | main_registro.cpp | [*] F_registro_array.cpp | Lib_registro_array.h

Project_personaarray

```cpp
#include <iostream>
#include "Lib_registro_array.h"

using namespace std;

int main(){

    Registro r;

    Persona personas[3];
    Auto autos[3];

    r.capturarPersonas(personas,3);
    r.capturarAutos(autos,3);

    r.mostrarPersonas(personas,3);
    r.mostrarAutos(autos,3);

    return 0;
}
```

D:\IPN 2025\TERCER SEMESTI

```
Apellido paterno: Entzana
Apellido materno: Garduño
Genero: Hombre
Edad: 30

Auto 1
Precio: 60000
Anio: 2010

Auto 2
Precio: 650000
Anio: 2013

Auto 3
Precio: 200000
Anio: 2024

Lista de Personas
Rodrigo Murata Abreu Genero: Hombre Edad: 19
Ian Van Scoit Genero: Utriz Edad: 19
Yventz Entzana Garduño Genero: Hombre Edad: 30

Lista de Autos
Auto 1 -> Precio: 60000 Anio: 2010
Auto 2 -> Precio: 650000 Anio: 2013
Auto 3 -> Precio: 200000 Anio: 2024

--------------------------------
Process exited after 108.2 seconds with return value 0
Presione una tecla para continuar . . .
```

14. LISTA DE PERSONAS CON PUNTERO AL ARREGLO

El siguiente código se desarrolló utilizando la estructura de archivos solicitada por el maestro:

- Programa Principal.
- Programa de Funciones.
- Librería.

Se utilizó una clase llamada “ListaPersonas”, en la cual se declararon las funciones necesarias para capturar y mostrar la información de una lista de personas.

Se utilizó la clase Persona del programa anterior.

Para acceder a cada elemento se utilizó la expresión:

(p+i)

y para acceder a los atributos de cada persona se utilizó:

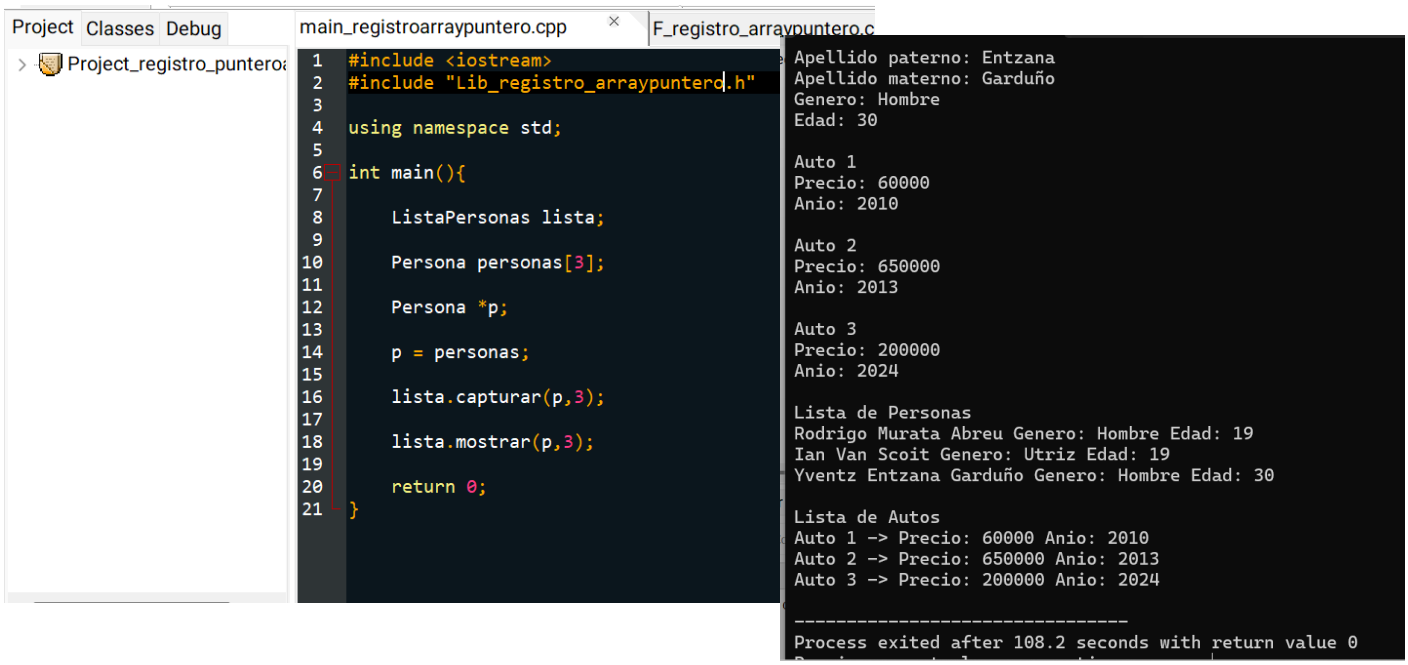
(p+i)->nombre

(p+i)->edad

Se utilizaron ciclos for para recorrer la lista y capturar los datos mediante cin, así como para mostrar la información almacenada mediante cout.

Durante el desarrollo del programa se presentaron algunos errores comunes, entre ellos:

- Confundir el acceso directo p[i] con el acceso indirecto (p+i).
- Olvidar utilizar el operador -> al acceder a los atributos del objeto apuntado por el puntero.
- No asignar correctamente el puntero al arreglo de personas.



15. FACTORIAL Y SERIE DE FIBONACCI CON FUNCIONES Fibo() Y Fibo2()

El siguiente programa se desarrolló utilizando la estructura de archivos solicitada por el maestro:

- Programa Principal.
- Programa de Funciones.
- Librería.

Se utilizó una clase llamada “Series”, en la cual se declararon las funciones necesarias para calcular el factorial de un número y generar la serie de Fibonacci.

Para la serie de Fibonacci se utilizaron dos funciones:

- Fibo(int n)
- Fibo2(int n)

La función Fibo(int n) calcula el valor de Fibonacci utilizando recursividad, donde cada término se obtiene sumando los dos términos anteriores.

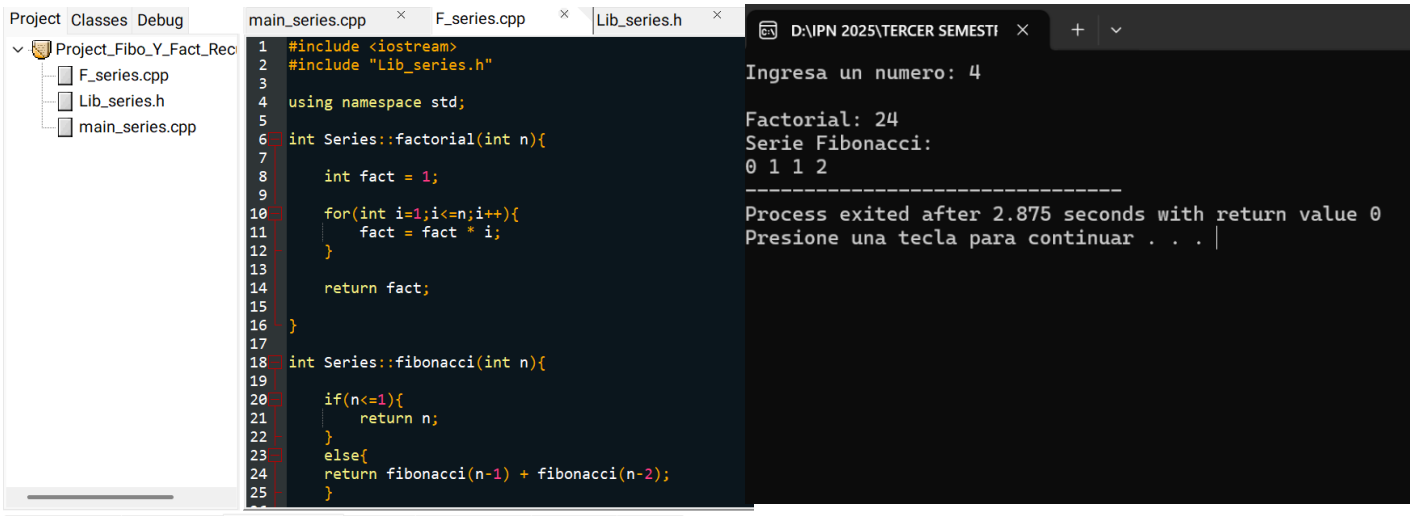
La función Fibo2(int n) se encarga de mostrar la serie completa, utilizando un ciclo for que llama repetidamente a la función Fibo() para calcular cada término de la serie.

También se incluyó la función factorial(int n), la cual calcula el factorial de un número utilizando un ciclo for, multiplicando todos los números desde 1 hasta el valor ingresado.

En el programa principal se solicita al usuario ingresar un número mediante cin. Posteriormente se calcula el factorial del número y se imprime en pantalla. Después se llama a la función Fibo2() para mostrar la serie de Fibonacci correspondiente.

Durante el desarrollo del programa se presentaron algunos errores comunes, entre ellos:

- Confundir la condición base en la función recursiva de Fibonacci.
- No llamar correctamente la función Fibo() dentro de Fibo2().
- Olvidar declarar correctamente las funciones dentro de la librería.



16. FRACTALES: TRIÁNGULO DE SIERPINSKI Y POLVO DE CANTOR

El siguiente programa se desarrolló utilizando la **estructura de archivos solicitada por el maestro**, la cual consiste en:

- Programa Principal.
- Programa de Funciones.
- Librería.

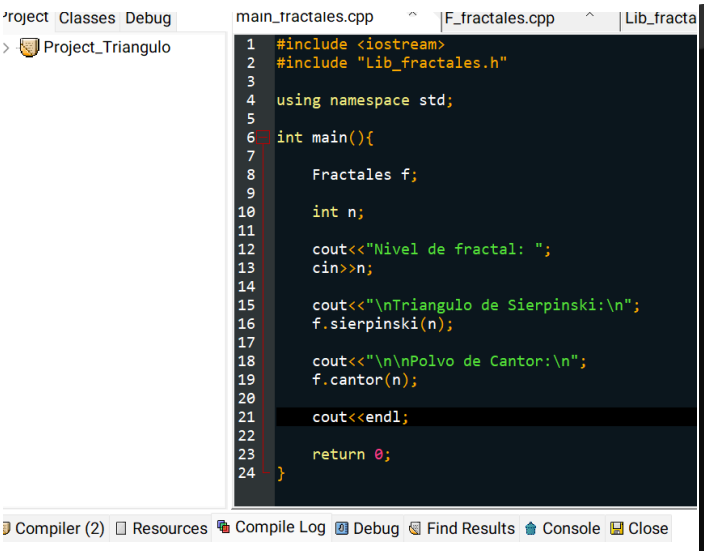
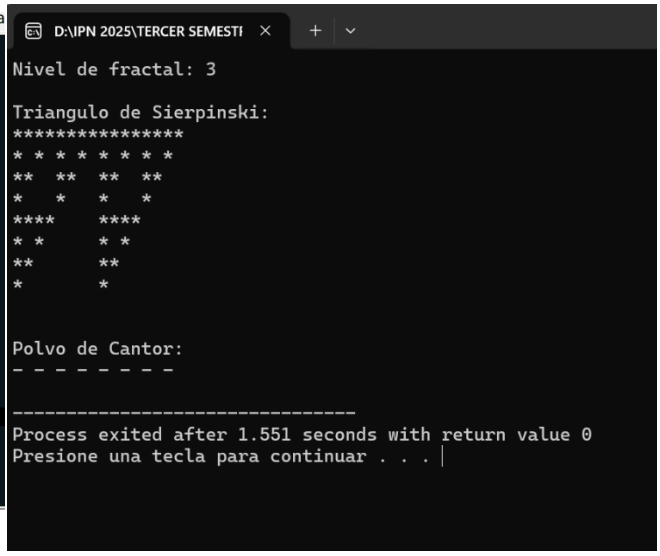
Se utilizó una **clase llamada "Fractales"**, en la cual se declararon las funciones necesarias para generar dos fractales matemáticos: **el triángulo de Sierpinski** y **el polvo de Cantor**.

Ambos fractales se generan mediante **recursividad**, es decir, una función que se llama a sí misma reduciendo el problema hasta llegar a un **caso base**.

La función sierpinski(int n) genera el patrón del triángulo de Sierpinski. En el caso base, cuando n = 0, se imprime un carácter *. En los demás casos la función se llama a sí misma dos veces, generando el patrón fractal.

La función cantor(int n) genera el patrón del **polvo de Cantor**, el cual también se basa en un proceso recursivo donde cada segmento se divide en partes más pequeñas separadas por espacios.

En el **programa principal** se solicita al usuario ingresar el nivel del fractal mediante cin. Posteriormente se llaman las funciones correspondientes para generar cada fractal y se muestran los resultados en pantalla utilizando cout.

Durante el desarrollo del programa se presentaron algunos errores comunes, entre ellos:

- No establecer correctamente el **caso base de la recursividad**, lo que puede provocar recursión infinita.
- Confundir el orden de las llamadas recursivas dentro de las funciones.
- No declarar correctamente las funciones dentro de la librería antes de utilizarlas.